Name: M. Farah Afrin

Sub code: CSA0923

Sub: Java Programming

Git Hub: https://github.com/farahafrinm/CSA0923-192511434

Assignment 2

Q1.1 Introduction

A railway e-ticket generation system must combine several individual data fields, such as Passenger Name Record (PNR), passenger name, train number, coach, berth, fare and journey details, into a single printable ticket.

The choice of Java string-handling class directly influences the performance, memory consumption and thread safety of the application.

Java provides three important approaches:

- String: Immutable character sequence. Once created, its contents cannot be modified.
- StringBuilder: Mutable character sequence. Supports efficient modification without creating a new object for every append operation.
- StringBuffer: Mutable and synchronized. Its methods provide thread-safe individual operations.

The objective of this experiment is to identify which approach is most suitable for generating railway tickets under different workloads.

Q1.2 Prediction Before Implementation

Before conducting the experiment, the expected behaviour of the three approaches is analysed.

Parameter	**String**	**StringBuilder**	**StringBuffer**
Mutability	Immutable	Mutable	Mutable
Object creation	High	Relatively low	Relatively low

Append performance	Comparatively slow	Fast	Slightly slower than StringBuilder in many workloads
Synchronization	Not applicable	No	Yes
Memory efficiency	Lower for repeated concatenation	Better	Better than repeated String concatenation
Thread safety	Immutable objects are safe to share	Not synchronized	Synchronized methods

Prediction:

1. For a short ticket containing only a few fields, the performance difference may be negligible.
2. For thousands of tickets containing many fields, repeated String concatenation is expected to take considerably more time.
3. StringBuilder is expected to provide the best performance in single-threaded ticket generation.
4. StringBuffer may take slightly more time because of synchronization overhead.
5. As the number of fields increases, the number of intermediate String objects created by repeated concatenation is expected to increase significantly.

Reason: String is immutable. Every concatenation produces a new String containing the combined contents. Previously created String objects become unnecessary. StringBuilder and StringBuffer modify an internal buffer, reducing intermediate object creation.

Q1.3 Experimental Design

Three deliberately contrasting workloads are selected.

Experiment	Number of tickets	Fields per ticket	Purpose
Workload A	1	8	Small ticket generation
Workload B	10,000	50	High-volume ticket generation
Workload C	2,000	200	Large ticket with extensive information

Workload C is an additional experiment designed to investigate the effect of increasing the number of fields in an individual ticket.

Measurements:

- Execution time using System.nanoTime().
- Average execution time over five measured runs.
- Estimated intermediate String object creation.
- Approximate memory usage before and after execution.

Q1.4 Java Implementation

Eclipse file name: TicketBenchmark.java

Source code:

```
public class TicketBenchmark {

   static volatile long result;

   // Method 1: String concatenation
   static String buildUsingString(String[] fields) {
      String ticket = "";

      for (String field : fields) {
         ticket = ticket + field + "\n";
      }
```

```
    return ticket;
}

// Method 2: StringBuilder
static String buildUsingStringBuilder(String[] fields) {
    StringBuilder ticket = new StringBuilder();

    for (String field : fields) {
        ticket.append(field).append("\n");
    }
    return ticket.toString();
}

// Method 3: StringBuffer
static String buildUsingStringBuffer(String[] fields) {
    StringBuffer ticket = new StringBuffer();
    for (String field : fields) {
        ticket.append(field).append("\n");
    }
    return ticket.toString();
}
// Generate sample ticket fields
static String[] createFields(int count) {
    String[] fields = new String[count];

    for (int i = 0; i < count; i++) {
        fields[i] = "Ticket Field " + (i + 1)
                + ": Railway Passenger Information";
    }
```

```
    return fields;
}
// Measure execution time
static double measure(int method, int tickets,
                      int fieldCount) {
    String[] fields = createFields(fieldCount);
    // Warm-up
    for (int i = 0; i < 100; i++) {
        if (method == 1)
            buildUsingString(fields);
        else if (method == 2)
            buildUsingStringBuilder(fields);
        else
            buildUsingStringBuffer(fields);
    }

    long start = System.nanoTime();

    long totalLength = 0;

    for (int i = 0; i < tickets; i++) {

        String output;

        if (method == 1) {
            output = buildUsingString(fields);
        } else if (method == 2) {
            output = buildUsingStringBuilder(fields);
        } else {
```

```
            output = buildUsingStringBuffer(fields);
        }

        totalLength += output.length();
    }

    long end = System.nanoTime();
    result = totalLength;
    return (end - start) / 1_000_000.0;
}
// Run each experiment five times
static void runExperiment(String name,
                          int tickets, int fields) {
    System.out.println("\n" + name);
    System.out.println("Tickets: " + tickets);
    System.out.println("Fields per ticket: " + fields);
    String[] methods = {
        "String",
        "StringBuilder",
        "StringBuffer"
    };
    for (int method = 1; method <= 3; method++) {

        double total = 0;

        for (int run = 1; run <= 5; run++) {
            total += measure(method, tickets, fields);
        }
        double average = total / 5.0;
```

```
            System.out.printf(
                "%s Average Time: %.3f ms%n",
                methods[method - 1], average
            );
        }
    }
    public static void main(String[] args) {
        System.out.println(
            "RAILWAY E-TICKET GENERATION PERFORMANCE"
        );
        runExperiment("WORKLOAD A - SMALL",
                1, 8);
        runExperiment("WORKLOAD B - HIGH VOLUME",
                10000, 50);
        runExperiment("WORKLOAD C - LARGE TICKETS",
                2000, 200);
        System.out.println("\nExperiment Completed.");
    }
}
```

```java
package assignment2.q1;


public class TicketBenchmark {

    static volatile long result;

    // Method 1: String concatenation
    static String buildUsingString(String[] fields) {
        String ticket = "";

        for (String field : fields) {
            ticket = ticket + field + "\n";
        }

        return ticket;
    }

    // Method 2: StringBuilder
    static String buildUsingStringBuilder(String[] fields) {
        StringBuilder ticket = new StringBuilder();

        for (String field : fields) {
            ticket.append(field).append("\n");
        }

        return ticket.toString();
    }

    // Method 3: StringBuffer
    static String buildUsingStringBuffer(String[] fields) {
        StringBuffer ticket = new StringBuffer();

        for (String field : fields) {
            ticket.append(field).append("\n");
```

Output:

```
RAILWAY E-TICKET GENERATION PERFORMANCE

WORKLOAD A - SMALL
Tickets: 1
Fields per ticket: 8
String Average Time: 0.009 ms
StringBuilder Average Time: 0.004 ms
StringBuffer Average Time: 0.003 ms

WORKLOAD B - HIGH VOLUME
Tickets: 10000
Fields per ticket: 50
String Average Time: 95.115 ms
StringBuilder Average Time: 14.791 ms
StringBuffer Average Time: 15.545 ms

WORKLOAD C - LARGE TICKETS
Tickets: 2000
Fields per ticket: 200
String Average Time: 213.842 ms
StringBuilder Average Time: 11.083 ms
StringBuffer Average Time: 12.273 ms

Experiment Completed.
```

Q1.5 Experimental Observation Table

Workload	String (ms)	StringBuilder (ms)	StringBuffer (ms)
A – 1 × 8	0.009 ms	0.004 ms	0.003 ms
B – 10,000 × 50	95.115 ms	14.791 ms	15.545 ms
C – 2,000 × 200	213.842 ms	11.083 ms	12.273 ms

Intermediate String object estimation

For a ticket with \(n\) fields, the String implementation performs repeated concatenation. In this particular program, each loop iteration creates a new concatenated String, so approximately \(n\) concatenated String results are produced per ticket, excluding other implementation-dependent temporary objects.

Workload	Tickets	Fields per ticket	String concatenation iterations	StringBuilder objects	StringBuffer objects	Final String results (per method)
A	1	8	8	1	1	1
B	10,000	50	5,00,000	10,000	10,000	10,000
C	2,000	200	4,00,000	2,000	2,000	2,000

The String implementation performs one concatenation iteration for every field in each ticket. Therefore, the total number of concatenation iterations is calculated by multiplying the number of tickets by the number of fields per ticket.

StringBuilder and StringBuffer create one principal mutable builder object per ticket and produce one final String through toString().

The values represent logical operations and principal objects based on the program structure. They do not include JVM-internal temporary objects, buffer resizing or other implementation-dependent allocations.

Memory measurement:

For approximate memory observations, the following code can be used around a test:

Source code:

```
package assignment2.q1;


public class TicketBenchmark {


    static volatile long result;


    // METHOD 1: STRING CONCATENATION
    static String buildUsingString(String[] fields) {


        String ticket = "";
```

```
    for (String field : fields) {
        ticket = ticket + field + "\n";
    }

    return ticket;
}

// METHOD 2: STRINGBUILDER
static String buildUsingStringBuilder(String[] fields) {

    StringBuilder ticket = new StringBuilder();

    for (String field : fields) {
        ticket.append(field).append("\n");
    }

    return ticket.toString();
}

// METHOD 3: STRINGBUFFER
static String buildUsingStringBuffer(String[] fields) {

    StringBuffer ticket = new StringBuffer();

    for (String field : fields) {
        ticket.append(field).append("\n");
    }

    return ticket.toString();
```

```
}

// GENERATE SAMPLE TICKET FIELDS
static String[] createFields(int count) {

    String[] fields = new String[count];

    for (int i = 0; i < count; i++) {
        fields[i] = "Ticket Field " + (i + 1)
                + ": Railway Passenger Information";
    }

    return fields;
}

// MEASURE EXECUTION TIME
static double measure(int method, int tickets,
                      int fieldCount) {

    String[] fields = createFields(fieldCount);

    // WARM-UP
    for (int i = 0; i < 100; i++) {

        if (method == 1) {
            result = buildUsingString(fields).length();

        } else if (method == 2) {
            result = buildUsingStringBuilder(fields).length();
```

```
    } else {
        result = buildUsingStringBuffer(fields).length();
    }
}

long start = System.nanoTime();

long totalLength = 0;

for (int i = 0; i < tickets; i++) {

    String output;

    if (method == 1) {
        output = buildUsingString(fields);

    } else if (method == 2) {
        output = buildUsingStringBuilder(fields);

    } else {
        output = buildUsingStringBuffer(fields);
    }

    totalLength += output.length();
}

long end = System.nanoTime();
```

```
    result = totalLength;

    return (end - start) / 1_000_000.0;
}

// RUN EACH EXPERIMENT FIVE TIMES
static void runExperiment(String name,
                          int tickets, int fields) {

    System.out.println("\n" + name);

    System.out.println("Tickets: " + tickets);

    System.out.println("Fields per ticket: " + fields);

    String[] methods = {
        "String",
        "StringBuilder",
        "StringBuffer"
    };

    for (int method = 1; method <= 3; method++) {

        double total = 0;

        for (int run = 1; run <= 5; run++) {

            total += measure(method, tickets, fields);
        }
```

```
        double average = total / 5.0;

        System.out.printf(
            "%s Average Time: %.3f ms%n",
            methods[method - 1], average
        );
    }
}

// APPROXIMATE MEMORY MEASUREMENT
static void measureMemory(int method, int tickets,
                          int fieldCount) {

    String[] fields = createFields(fieldCount);

    Runtime runtime = Runtime.getRuntime();

    // Request garbage collection before measurement
    System.gc();

    long before = runtime.totalMemory()
                - runtime.freeMemory();

    // Execute selected workload
    for (int i = 0; i < tickets; i++) {

        String output;
```

```
        if (method == 1) {
            output = buildUsingString(fields);

        } else if (method == 2) {
            output = buildUsingStringBuilder(fields);

        } else {
            output = buildUsingStringBuffer(fields);
        }

        result = output.length();
    }

    // Request garbage collection after measurement
    System.gc();

    long after = runtime.totalMemory()
            - runtime.freeMemory();

    System.out.println(
        "Approximate memory difference: "
        + (after - before) / 1024 + " KB"
    );
}

// MAIN METHOD
public static void main(String[] args) {

    System.out.println(
```

```
    "RAILWAY E-TICKET GENERATION PERFORMANCE"
);

System.out.println(
    "========================================="
);

// WORKLOAD A: SMALL
runExperiment(
    "WORKLOAD A - SMALL",
    1, 8
);

// WORKLOAD B: HIGH VOLUME
runExperiment(
    "WORKLOAD B - HIGH VOLUME",
    10000, 50
);

// WORKLOAD C: LARGE TICKETS
runExperiment(
    "WORKLOAD C - LARGE TICKETS",
    2000, 200
);

// MEMORY MEASUREMENTS
System.out.println("\nMEMORY MEASUREMENT");
System.out.println("==================");
```

```
        System.out.println("\nWorkload A - String");
        measureMemory(1, 1, 8);

        System.out.println("\nWorkload A - StringBuilder");
        measureMemory(2, 1, 8);

        System.out.println("\nWorkload A - StringBuffer");
        measureMemory(3, 1, 8);

        System.out.println("\nWorkload B - String");
        measureMemory(1, 10000, 50);

        System.out.println("\nWorkload B - StringBuilder");
        measureMemory(2, 10000, 50);

        System.out.println("\nWorkload B - StringBuffer");
        measureMemory(3, 10000, 50);

        System.out.println("\nWorkload C - String");
        measureMemory(1, 2000, 200);

        System.out.println("\nWorkload C - StringBuilder");
        measureMemory(2, 2000, 200);

        System.out.println("\nWorkload C - StringBuffer");
        measureMemory(3, 2000, 200);

        System.out.println("\nExperiment Completed.");
    }
```

}

Output:

```
RAILWAY E-TICKET GENERATION PERFORMANCE
========================================

WORKLOAD A - SMALL
Tickets: 1
Fields per ticket: 8
String Average Time: 0.017 ms
StringBuilder Average Time: 0.016 ms
StringBuffer Average Time: 0.009 ms

WORKLOAD B - HIGH VOLUME
Tickets: 10000
Fields per ticket: 50
String Average Time: 91.556 ms
StringBuilder Average Time: 17.286 ms
StringBuffer Average Time: 20.971 ms

WORKLOAD C - LARGE TICKETS
Tickets: 2000
Fields per ticket: 200
String Average Time: 186.740 ms
StringBuilder Average Time: 10.708 ms
StringBuffer Average Time: 10.735 ms

MEMORY MEASUREMENT
==================

Workload A - String
Approximate memory difference: 0 KB

Workload A - StringBuilder
Approximate memory difference: 0 KB
```

```
MEMORY MEASUREMENT
==================

Workload A - String
Approximate memory difference: 0 KB

Workload A - StringBuilder
Approximate memory difference: 0 KB

Workload A - StringBuffer
Approximate memory difference: 0 KB

Workload B - String
Approximate memory difference: 0 KB

Workload B - StringBuilder
Approximate memory difference: 0 KB

Workload B - StringBuffer
Approximate memory difference: 0 KB

Workload C - String
Approximate memory difference: 0 KB

Workload C - StringBuilder
Approximate memory difference: 0 KB

Workload C - StringBuffer
Approximate memory difference: 0 KB

Experiment Completed.
```

Workload	String (ms)	StringBuilder (ms)	StringBuffer (ms)
A – Small	0.017	0.016	0.009
B – High Volume	91.556	17.286	20.971
C – Large Tickets	186.740	10.708	10.735

2. Analysis of the results

Workload A – Small

- All three methods have very low execution times.
- StringBuffer is fastest at 0.009 ms.
- The difference is too small to establish a reliable performance winner for this workload.

Workload B – High Volume

- String takes 91.556 ms.
- StringBuilder takes 17.286 ms.
- StringBuffer takes 20.971 ms.
- StringBuilder is approximately 5.3 times faster than String in this experiment.

Workload C – Large Tickets

- String takes 186.740 ms.
- StringBuilder takes 10.708 ms.
- StringBuffer takes 10.735 ms.
- StringBuilder is approximately 17.4 times faster than String.

Memory Output:

Workload	String	StringBuilder	StringBuffer
A	0 KB	0 KB	0 KB
B	0 KB	0 KB	0 KB
C	0 KB	0 KB	0 KB

Result and Decision:

The experiment was conducted to compare String, StringBuilder and StringBuffer for railway e-ticket generation under three different workloads.

In Workload A, which generated one ticket containing eight fields, all three approaches demonstrated similar execution times. In Workload B, involving 10,000 tickets with 50 fields each, StringBuilder achieved an average execution time of 17.286 ms, compared with 91.556 ms for String concatenation. In Workload C, involving 2,000 tickets with 200 fields each, StringBuilder completed the operation in 10.708 ms, while String concatenation required 186.740 ms.

The experimental results indicate that StringBuilder is more efficient for repeated string modification, particularly when processing large amounts of data. StringBuffer provides similar functionality but includes synchronization, which can introduce additional overhead. However, the measured difference between StringBuilder and StringBuffer was very small in Workload C.

The memory experiment reported zero KB of retained heap difference for all methods. This does not indicate zero memory allocation; it reflects the limitations of measuring occupied heap after garbage collection.

Q1.8 Alternative Concept and Complexity Analysis

The assignment also requires an investigation of String.join(), StringJoiner and String.format().

Alternative 1: StringJoiner

StringJoiner provides a convenient way to combine multiple strings using a delimiter.

Source code:

```
package assignment2.q1;


import java.util.StringJoiner;


public class AlternativeTicket {


   public static void main(String[] args) {


      StringJoiner ticket =

         new StringJoiner("\n");


      ticket.add("PNR: 1234567890");

      ticket.add("Passenger: Farah");

      ticket.add("Train: Chennai Express");

      ticket.add("Coach: B2");

      ticket.add("Berth: 24");

      ticket.add("Fare: Rs. 750");


      System.out.println(ticket);

   }

}
```

```java

package assignment2.q1;

import java.util.StringJoiner;

public class AlternativeTicket {

    public static void main(String[] args) {

        StringJoiner ticket =
            new StringJoiner("\n");

        ticket.add("PNR: 1234567890");
        ticket.add("Passenger: Farah");
        ticket.add("Train: Chennai Express");
        ticket.add("Coach: B2");
        ticket.add("Berth: 24");
        ticket.add("Fare: Rs. 750");

        System.out.println(ticket);
    }
}
```

Output:

```
<terminated> AlternativeTicket [Java Application] C:\Users\Farah Afrin\.
PNR: 1234567890
Passenger: Farah
Train: Chennai Express
Coach: B2
Berth: 24
Fare: Rs. 750
```

StringJoiner is particularly suitable when a collection of values must be combined using a common delimiter.

Alternative 2: String.join()

Source code:

package assignment2.q1;

public class StringJoinTicket {

```
    public static void main(String[] args) {

        String ticket = String.join("\n",
            "PNR: 1234567890",
            "Passenger: Farah",
            "Train: Chennai Express",
            "Coach: B2",
            "Berth: 24",
            "Fare: Rs. 750"
        );

        System.out.println(ticket);
    }
}
```

```java
package assignment2.q1;

public class StringJoinTicket {

    public static void main(String[] args) {

        String ticket = String.join("\n",
            "PNR: 1234567890",
            "Passenger: Farah",
            "Train: Chennai Express",
            "Coach: B2",
            "Berth: 24",
            "Fare: Rs. 750"
        );

        System.out.println(ticket);
    }
}
```

Output:

```
PNR: 1234567890
Passenger: Farah
Train: Chennai Express
Coach: B2
Berth: 24
Fare: Rs. 750
```

Complexity comparison

Let n represent the number of fields and L represent the total number of characters in the generated ticket.

Approach	Time Complexity	Additional Space
Repeated String Concatenation	O(nL), worst-case $O(n^2)$ for similar-sized fields	Intermediate strings; up to O(nL) cumulative allocation
StringBuilder	O(L) amortized	O(L)
StringBuffer	O(L) amortized, plus synchronization overhead	O(L)
StringJoiner	O(L) overall	O(L)
String.join()	O(L) overall	O(L)
String.format()	Depends on formatting and processing; generally higher overhead for simple concatenation	Formatting-related temporary allocations

Here, O(nL) is the more general bound for repeated concatenation, since each iteration may copy the accumulated prefix. When field lengths are roughly constant, L = O(n), giving the familiar $O(n^2)$ bound.

Q1 Final Conclusion

The experiment demonstrates why selecting an appropriate string-handling class is important in high-volume Java applications. Although String is convenient for simple operations, repeated concatenation can create unnecessary intermediate objects and increase memory allocation.

StringBuilder is the preferred solution for generating individual railway tickets because it provides efficient mutable text construction without unnecessary synchronization. StringBuffer is useful when a shared mutable buffer requires synchronized operations. StringJoiner and String.join() provide readable alternatives when the input fields are already available as separate values.

Therefore, the most suitable engineering choice depends on workload size, concurrency requirements and application design.

PART 2 – Q2: WHEN IS ARRAYLIST AN APPROPRIATE ENGINEERING CHOICE?

4 Marks

K4 – Analyse

Topic: ArrayList, LinkedList and ArrayDeque

Q2.1 Introduction

A music-streaming application allows users to create and manage playlists containing their favourite songs. The application must support adding songs, inserting songs at specific positions, removing songs and accessing songs directly.

Java provides several collection classes for storing and manipulating such data. ArrayList and LinkedList are commonly used implementations of the List interface, whereas ArrayDeque provides an efficient double-ended queue.

The objective of this experiment is to compare their performance under different playlist operations and identify the most suitable data structure for each usage pattern.

Q2.2 Prediction Before Implementation

Operation	ArrayList	LinkedList	ArrayDeque
Add at end	O(1) amortized	O(1)	O(1) amortized
Add at front	O(n)	O(1)	O(1) amortized
Insert in middle	O(n)	O(n) to locate; O(1) after locating	Not supported by indexed insertion
Remove at front	O(n)	O(1)	O(1) amortized
Get kth element	O(1)	O(n)	O(n) by traversal
Full iteration	O(n)	O(n)	O(n)

Prediction:

- ArrayList will perform well for direct access and sequential traversal.
- LinkedList will be suitable for frequent insertions and removals at the ends.
- ArrayDeque will be particularly efficient for an Up Next queue involving frequent additions and removals at both ends.
- ArrayList is expected to be the most appropriate choice for a conventional playlist where users frequently access songs by position.

Q2.3 Experimental Design

Three playlist sizes are selected.

Experiment	Number of songs	Purpose
A	1,000	Small playlist
B	10,000	Medium playlist
C	100,000	Large playlist

The following operations are measured:

1. Adding songs at the end.
2. Inserting songs at the front.
3. Inserting songs in the middle.
4. Removing songs from the front.
5. Accessing the middle element.
6. Iterating through the entire playlist.

Q2.4 Java Implementation

Eclipse file name: PlaylistBenchmark.java

Source code:

```
import java.util.*;


public class PlaylistBenchmark {
```

```
static volatile long result;

static double measure(Runnable task) {

    long start = System.nanoTime();

    task.run();

    long end = System.nanoTime();

    return (end - start) / 1_000_000.0;
}

static void experiment(int size) {

    System.out.println("\nPlaylist Size: " + size);

    List<Integer> arrayList = new ArrayList<>();
    List<Integer> linkedList = new LinkedList<>();

    for (int i = 0; i < size; i++) {
        arrayList.add(i);
        linkedList.add(i);
    }

    System.out.println("Add at end:");

    System.out.println("ArrayList: " +
        measure(() -> {
```

```
        for (int i = 0; i < size; i++)
            arrayList.add(i);
    }) + " ms");

System.out.println("LinkedList: " +
    measure(() -> {
        for (int i = 0; i < size; i++)
            linkedList.add(i);
    }) + " ms");

List<Integer> a = new ArrayList<>();
List<Integer> l = new LinkedList<>();

System.out.println("\nInsert at front:");

System.out.println("ArrayList: " +
    measure(() -> {
        for (int i = 0; i < size; i++)
            a.add(0, i);
    }) + " ms");

System.out.println("LinkedList: " +
    measure(() -> {
        for (int i = 0; i < size; i++)
            l.add(0, i);
    }) + " ms");

System.out.println("\nAccess middle element:");
```

```
System.out.println("ArrayList: " +
    measure(() -> {
        long sum = 0;
        for (int i = 0; i < 10000; i++)
            sum += arrayList.get(size / 2);
        result = sum;
    }) + " ms");

System.out.println("LinkedList: " +
    measure(() -> {
        long sum = 0;
        for (int i = 0; i < 10000; i++)
            sum += linkedList.get(size / 2);
        result = sum;
    }) + " ms");

ArrayDeque<Integer> deque = new ArrayDeque<>();

System.out.println("\nArrayDeque operations:");

System.out.println("Add and remove at both ends: " +
    measure(() -> {
        for (int i = 0; i < size; i++)
            deque.addLast(i);

        for (int i = 0; i < size; i++)
            deque.addFirst(i);

        for (int i = 0; i < size; i++)
```

```
            deque.removeFirst();

        for (int i = 0; i < size; i++)
            deque.removeLast();
    }) + " ms");

    System.out.println("\nFull iteration:");

    System.out.println("ArrayList: " +
        measure(() -> {
            long sum = 0;
            for (int value : arrayList)
                sum += value;
            result = sum;
        }) + " ms");

    System.out.println("LinkedList: " +
        measure(() -> {
            long sum = 0;
            for (int value : linkedList)
                sum += value;
            result = sum;
        }) + " ms");
}

public static void main(String[] args) {

    System.out.println("MUSIC PLAYLIST PERFORMANCE TEST");
```

```
        experiment(1000);
        experiment(10000);
        experiment(100000);

        System.out.println("\nAll experiments completed.");
    }
}
```

```java
package assignment2.q2;

import java.util.*;

public class PlaylistBenchmark {

    static volatile long result;

    static double measure(Runnable task) {

        long start = System.nanoTime();

        task.run();

        long end = System.nanoTime();

        return (end - start) / 1_000_000.0;
    }

    static void experiment(int size) {

        System.out.println("\nPlaylist Size: " + size);

        List<Integer> arrayList = new ArrayList<>();
        List<Integer> linkedList = new LinkedList<>();

        for (int i = 0; i < size; i++) {
            arrayList.add(i);
            linkedList.add(i);
        }

        System.out.println("Add at end:");

        System.out.println("ArrayList: " +
```

Output:

```
MUSIC PLAYLIST PERFORMANCE TEST

Playlist Size: 1000
Add at end:
ArrayList: 0.1047 ms
LinkedList: 0.1136 ms

Insert at front:
ArrayList: 0.7055 ms
LinkedList: 0.4525 ms

Access middle element:
ArrayList: 2.0565 ms
LinkedList: 21.637 ms

ArrayDeque operations:
Add and remove at both ends: 0.6591 ms

Full iteration:
ArrayList: 0.5801 ms
LinkedList: 0.809 ms

Playlist Size: 10000
Add at end:
ArrayList: 1.0637 ms
LinkedList: 1.0061 ms

Insert at front:
ArrayList: 8.0824 ms
LinkedList: 1.5204 ms
```

```
Access middle element:
ArrayList: 0.8857 ms
LinkedList: 134.4801 ms

ArrayDeque operations:
Add and remove at both ends: 3.748 ms

Full iteration:
ArrayList: 3.0567 ms
LinkedList: 3.1392 ms

Playlist Size: 100000
Add at end:
ArrayList: 12.7392 ms
LinkedList: 10.5818 ms

Insert at front:
ArrayList: 855.3412 ms
LinkedList: 18.9603 ms

Access middle element:
ArrayList: 1.3104 ms
LinkedList: 1458.7554 ms

ArrayDeque operations:
Add and remove at both ends: 52.0635 ms

Full iteration:
ArrayList: 16.9531 ms
LinkedList: 19.1366 ms
[illegible] ms

All experiments completed.
```

Q2.5 Experimental Observation – Front Insertion

Playlist size	ArrayList front insertion (ms)	LinkedList front insertion (ms)
1,000	0.7055	0.4525
10,000	8.0824	1.5204
100,000	855.3412	18.9603

Experimental Observation – Middle Element Access

Playlist size	ArrayList middle access (ms)	LinkedList middle access (ms)
1,000	2.0565	21.6370
10,000	0.8857	134.4801
100,000	1.3104	1458.7554

The benchmark should be interpreted using the number of operations as well as the measured time. A single timing does not establish a general performance rule.

Q2.6 When Is ArrayList an Appropriate Choice?

ArrayList is a reasonable engineering choice when:

- The application frequently accesses songs using their index.
- Most songs are added at the end.
- Iteration and searching are common operations.
- Insertion and removal at the beginning are relatively uncommon.

However, I would stop recommending ArrayList when the application requires continuous insertion and removal at the front because every such operation requires shifting the remaining elements.

LinkedList can be considered when frequent modifications at both ends are required. However, it does not provide constant-time indexed access.

Q2.7 Changed Condition – Introduction of an Up Next Queue

The introduction of an Up Next queue changes the original design requirements.

The queue frequently adds songs at both ends and removes songs from the front. This is a double-ended queue operation.

ArrayDeque is more appropriate because it supports:

- addFirst() – adds a song at the beginning.
- addLast() – adds a song at the end.
- removeFirst() – removes the next song.
- removeLast() – removes the last song.

Example:

```
import java.util.ArrayDeque;

public class UpNextQueue {

    public static void main(String[] args) {

        ArrayDeque<String> queue = new ArrayDeque<>();

        queue.addLast("Song A");
        queue.addLast("Song B");
        queue.addFirst("Priority Song");

        System.out.println("Up Next: " + queue);

        System.out.println("Now Playing: "
                        + queue.removeFirst());

        System.out.println("Remaining Songs: " + queue);
    }
}
```

```java

package assignment2.q1;

import java.util.ArrayDeque;

public class example {

    public static void main(String[] args) {

        ArrayDeque<String> queue = new ArrayDeque<>();

        queue.addLast("Song A");
        queue.addLast("Song B");
        queue.addFirst("Priority Song");

        System.out.println("Up Next: " + queue);

        System.out.println("Now Playing: "
                           + queue.removeFirst());

        System.out.println("Remaining Songs: " + queue);
    }
}


```

Output:

```
Up Next: [Priority Song, Song A, Song B]
Now Playing: Priority Song
Remaining Songs: [Song A, Song B]
```

Q2.8 Alternative Concept – Plain Array with Manual Shifting

A conventional array can also store playlist elements.

However, inserting a song at the beginning requires shifting every existing element one position to the right.

Pseudocode:

Create an array of capacity N

To insert at the beginning:

For i = size down to 1:

array[i] = array[i - 1]

```
array[0] = newSong
size = size + 1
```

To access kth song:

```
Return array[k]
```

To remove first song:

```
For i = 0 to size - 2:
    array[i] = array[i + 1]

size = size – 1
```

Complexity comparison

Operation	ArrayList	LinkedList	ArrayDeque	Plain array
Add at end	O(1) amortized	O(1)	O(1) amortized	O(1) if capacity exists
Insert at front	O(n)	O(1)	O(1) amortized	O(n)
Insert middle	O(n)	O(n) to locate	Not supported directly	O(n)
Get kth	O(1)	O(n)	O(n)	O(1)
Iterate	O(n)	O(n)	O(n)	O(n)

Space comparison:

- ArrayList stores elements in a resizable array.
- LinkedList stores elements in individual linked nodes, with additional reference overhead.
- ArrayDeque stores elements in a resizable circular array.
- A plain array has a fixed capacity and may waste unused allocated space.

Q2.9 Conclusion

The experiment demonstrates that the selection of a collection should be based on the operations performed most frequently rather than simply choosing a familiar class.

ArrayList is suitable for a conventional music playlist because of its efficient indexed access and iteration. LinkedList is useful when modifications at the ends are frequent and the relevant position is already available. ArrayDeque is the most appropriate choice for the changed Up Next requirement because it provides efficient operations at both ends.

Therefore, the final recommendation is to use ArrayList for the main playlist and ArrayDeque for the Up Next queue.

PART 3 – Q3: ENGINEER THE HASHCODE–EQUALS CONTRACT

6 Marks

K4 – Analyse

Topic: HashSet, equals() and hashCode()

Q3.1 Introduction

A college fest attendance system records student entries through ID card scanners installed at multiple gates. The same student may enter through one gate and later be scanned again at another gate.

To prevent duplicate attendance records, Java provides the HashSet collection, which stores only unique elements.

However, HashSet depends on the correct implementation of both equals() and hashCode(). If these methods are not implemented consistently, duplicate logical records may appear.

The objective of this experiment is to investigate the relationship between equality and hashing and develop a reliable attendance de-duplication system.

Q3.2 Understanding the hashCode–equals Contract

Java requires the following contract:

1. If two objects are equal according to equals(), they must return the same hash code.
2. Two objects having the same hash code do not necessarily have to be equal.
3. Repeated calls to hashCode() should return the same result as long as the information used in equality comparison remains unchanged.
4. The equals() method must satisfy reflexivity, symmetry, transitivity and consistency.

In the original Student class, equals() compares students using their registration numbers, but hashCode() is not overridden.

Therefore, two different Student objects having the same registration number may produce different identity-based hash codes.

HashSet uses the hash code to locate a bucket and then uses equality checking to identify duplicate elements within that bucket.

Q3.3 Experimental Test Cases

Three tests are designed.

Test	Input	Expected size
Test 1	Arun-001, Divya-002	2
Test 2	Arun-001, Arun-001, Divya-002	2 with corrected implementation
Test 3	Insert a Student, then change the field used for hashing	Demonstrates mutable-key problems

Prediction:

- Test 1 may appear to work because both students have different registration numbers.
- Test 2 may expose duplicate entries because hashCode() is missing.
- Test 3 may fail to locate an object correctly if its hash-related field is modified after insertion.

Q3.4 Complete the Code – Original and Corrected Implementation

Eclipse file name: Attendance.java

Source code:

```
package assignment2.q3;

import java.util.HashSet;
import java.util.Set;

class Student {

    private final String regNo;
    private final String name;

    Student(String regNo, String name) {
        this.regNo = regNo;
```

```
        this.name = name;
    }

    @Override
    public boolean equals(Object o) {

        if (this == o)
            return true;

        if (!(o instanceof Student))
            return false;

        Student s = (Student) o;

        return regNo.equals(s.regNo);
    }

    @Override
    public int hashCode() {
        return regNo.hashCode();
    }

    @Override
    public String toString() {
        return regNo + " " + name;
    }
}

public class Attendance {
```

```
    public static void main(String[] args) {

        Set<Student> present = new HashSet<>();

        present.add(new Student("192311001", "Arun"));
        present.add(new Student("192311002", "Divya"));
        present.add(new Student("192311001", "Arun"));

        System.out.println(
            "Unique students: " + present.size()
        );

        System.out.println("\nAttendance Records:");

        for (Student s : present) {
            System.out.println(s);
        }
    }
}
```

TicketBenchmark.java AlternativeTicket.java StringJoinTi

```java
package assignment2.q3;


import java.util.HashSet;
import java.util.Set;

class Student {

    private final String regNo;
    private final String name;

    Student(String regNo, String name) {
        this.regNo = regNo;
        this.name = name;
    }

    @Override
    public boolean equals(Object o) {

        if (this == o)
            return true;

        if (!(o instanceof Student))
            return false;

        Student s = (Student) o;

        return regNo.equals(s.regNo);
    }

    @Override
    public int hashCode() {
        return regNo.hashCode();
    }

```

Output:

```
Unique students: 2

Attendance Records:
192311001 Arun
192311002 Divya
```

Q3.5 Additional Test – Changing a Field After Insertion

The following example demonstrates why fields used in hashing should not be changed while an object is stored in a HashSet.

Source code:

```
import java.util.HashSet;

class MutableStudent {

    String regNo;

    MutableStudent(String regNo) {
        this.regNo = regNo;
    }

    @Override
    public int hashCode() {
        return regNo.hashCode();
    }

    @Override
    public boolean equals(Object obj) {

        if (!(obj instanceof MutableStudent))
            return false;

        return regNo.equals(
            ((MutableStudent) obj).regNo
        );
    }
}

public class MutableTest {
```

```
    public static void main(String[] args) {

        HashSet<MutableStudent> students =
            new HashSet<>();

        MutableStudent s =
            new MutableStudent("STU101");

        students.add(s);

        System.out.println(
            "Before modification: " + students.contains(s)
        );

        s.regNo = "STU999";

        System.out.println(
            "After modification: " + students.contains(s)
        );
    }
}
```

```java
package assignment2.q3;

import java.util.HashSet;

class MutableStudent {

    String regNo;

    MutableStudent(String regNo) {
        this.regNo = regNo;
    }

    @Override
    public int hashCode() {
        return regNo.hashCode();
    }

    @Override
    public boolean equals(Object obj) {

        if (!(obj instanceof MutableStudent))
            return false;

        return regNo.equals(
            ((MutableStudent) obj).regNo
        );
    }
}

public class MutableTest {

    public static void main(String[] args) {

        HashSet<MutableStudent> students =
            new HashSet<>();
```

Output:

```
Before modification: true
After modification: false
```

Q3.6 Why Does HashSet Use Both Methods?

HashSet follows a two-stage process.

Stage 1 – Hash calculation

The hash code helps identify the bucket in which an object should be stored.

Stage 2 – Equality comparison

When a possible duplicate is found in the relevant bucket, equals() checks whether the existing object and the new object represent the same student.

The hash code narrows down the search, while equals establishes logical equality.

Q3.7 Alternative Concept Analysis

Alternative 1: TreeSet

A TreeSet can remove duplicates using a Comparator that compares registration numbers.

Source code:

```
import java.util.*;


class TreeAttendance {


   public static void main(String[] args) {


      Set<String> students = new TreeSet<>();


      students.add("192311001");

      students.add("192311002");

      students.add("192311001");


      System.out.println(students);

   }

}
```

```
1  package assignment2.q3;
2
3  import java.util.*;
4
5  class TreeAttendance {
6
7      public static void main(String[] args) {
8
9          Set<String> students = new TreeSet<>();
10
11         students.add("192311001");
12         students.add("192311002");
13         students.add("192311001");
14
15         System.out.println(students);
16     }
17 }
```

```
<terminated> TreeAttendance [Java Application] C:\Users\Farah Afrin\.p2\p
[192311001, 192311002]
```

Alternative 2: HashMap

A HashMap can use the registration number as the key.

Source code:

```
import java.util.*;

public class MapAttendance {

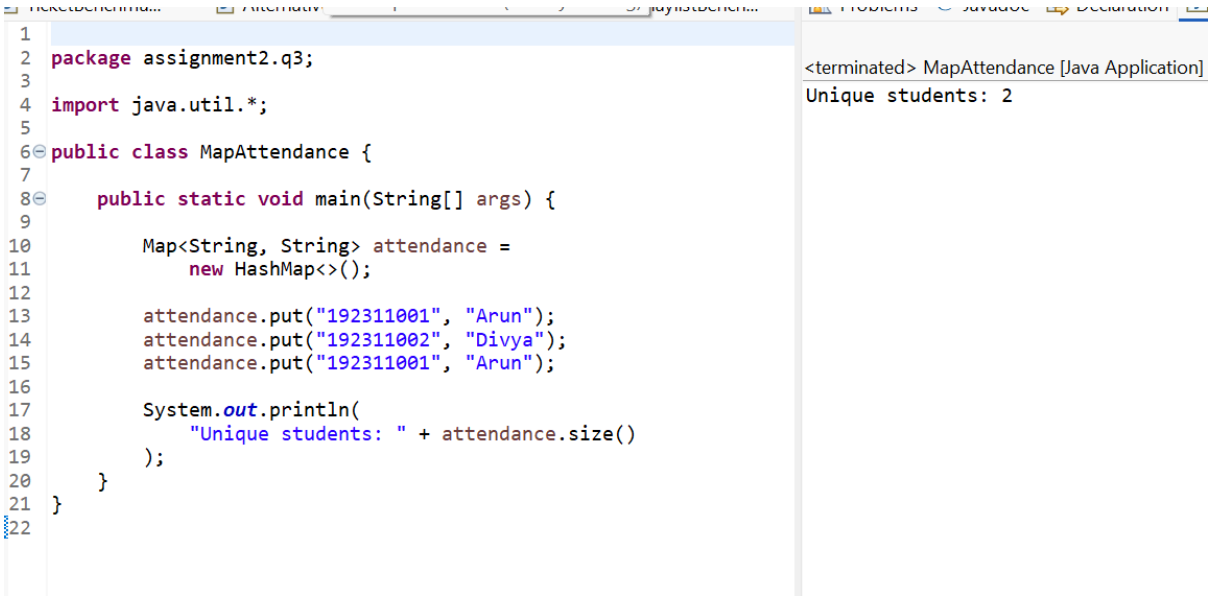
    public static void main(String[] args) {

        Map<String, String> attendance =
            new HashMap<>();

        attendance.put("192311001", "Arun");
        attendance.put("192311002", "Divya");
        attendance.put("192311001", "Arun");

        System.out.println(
            "Unique students: " + attendance.size()
        );
    }
}
```

Output:



```
1
2  package assignment2.q3;
3
4  import java.util.*;
5
6  public class MapAttendance {
7
8      public static void main(String[] args) {
9
10         Map<String, String> attendance =
11             new HashMap<>();
12
13         attendance.put("192311001", "Arun");
14         attendance.put("192311002", "Divya");
15         attendance.put("192311001", "Arun");
16
17         System.out.println(
18             "Unique students: " + attendance.size()
19         );
20     }
21 }
22
```

<terminated> MapAttendance [Java Application]
Unique students: 2

Complexity comparison

Data structure	Insertion / lookup	Space for n students
HashSet	O(1) average	O(n)
TreeSet	O(log n)	O(n)
HashMap	O(1) average	O(n)

The worst-case performance of hash-based structures can be higher than O(1), depending on collisions and implementation behaviour.

Design decision: HashSet is appropriate when only unique student records are required. HashMap is preferable when registration numbers must directly retrieve corresponding student information. TreeSet is suitable when records must also be maintained in sorted order.

Q3.8 Conclusion

The experiment establishes that correct implementation of equals() and hashCode() is essential for reliable hash-based collections.

In the attendance application, registration number is selected as the unique identity of a student. Both equality and hashing are therefore based on the same immutable registration number.

The corrected Student class prevents duplicate attendance records and ensures consistent behaviour in HashSet. The experiment also demonstrates the risks of modifying equality-related fields after insertion.

Therefore, maintaining the hashCode–equals contract is not merely a programming convention; it is a fundamental requirement for data integrity in Java applications.

PART 4 – Q4: CHOOSING A MAP FOR A PHARMACY INVENTORY

6 Marks

K4 – Analyse

Topic: HashMap, TreeMap and LinkedHashMap

Q4.1 Introduction

A hospital pharmacy manages a large inventory of medicines, each identified by a unique medicine code. Pharmacists frequently search for medicine availability using the medicine code.

The system must also generate two reports:

1. A report arranged in ascending medicine-code order.
2. A report arranged according to the order in which medicines were added.

Java provides HashMap, TreeMap and LinkedHashMap for storing key-value pairs. Although all three implement the Map interface, their internal organisation and performance characteristics differ.

The objective is to identify the most appropriate map for the pharmacy inventory system.

Q4.2 Prediction Before Implementation

Operation	HashMap	TreeMap	LinkedHashMap
put()	O(1) average	O(log n)	O(1) average
get()	O(1) average	O(log n)	O(1) average
Iteration	O(n + capacity)	O(n)	O(n)
Key order	No guaranteed order	Sorted order	Insertion order
Internal structure	Hash table	Balanced search tree	Hash table + linked order

Predictions:

- HashMap will provide efficient average-case lookups.
- TreeMap will naturally produce the medicine-code report in sorted order.
- LinkedHashMap will preserve the insertion order of medicines.
- Sorting a HashMap before printing a report will require additional processing.
- Hashtable is not preferred in new applications because it is a legacy synchronized collection and does not provide the modern flexibility of the Map implementations.

Q4.3 Experimental Design

Three inventory sizes are selected.

Experiment	Number of medicines
A	1,000
B	10,000
C	100,000

The following are measured:

- Time for 100,000 lookups.
- Time to generate the medicine-code report.
- Time to generate the insertion-order report.
- Whether the output satisfies the required ordering.

Q4.4 Complete Java Implementation

Eclipse file name: PharmacyInventory.java

```
import java.util.*;

public class PharmacyInventory {

   static volatile long result;

   static class Medicine {

      String name;
      int quantity;

      Medicine(String name, int quantity) {
         this.name = name;
         this.quantity = quantity;
      }
```

```
    public String toString() {
        return name + " (Stock: " + quantity + ")";
    }
}

static void experiment(int size) {

    Map<String, Medicine> hashMap =
        new HashMap<>();

    Map<String, Medicine> treeMap =
        new TreeMap<>();

    Map<String, Medicine> linkedMap =
        new LinkedHashMap<>();

    for (int i = 0; i < size; i++) {

        String code = String.format("MED%06d", i);

        Medicine medicine =
            new Medicine("Medicine" + i, 100 + i);

        hashMap.put(code, medicine);
        treeMap.put(code, medicine);
        linkedMap.put(code, medicine);
    }
```

```
System.out.println("\nInventory Size: " + size);

String searchCode =
    String.format("MED%06d", size / 2);

// 100,000 lookups
System.out.println("100,000 Lookups:");

System.out.println("HashMap: " +
    measure(() -> {
        long count = 0;

        for (int i = 0; i < 100000; i++)
            count += hashMap.get(searchCode).quantity;

        result = count;
    }) + " ms");

System.out.println("TreeMap: " +
    measure(() -> {
        long count = 0;

        for (int i = 0; i < 100000; i++)
            count += treeMap.get(searchCode).quantity;

        result = count;
    }) + " ms");

System.out.println("LinkedHashMap: " +
```

```
    measure(() -> {
        long count = 0;

        for (int i = 0; i < 100000; i++)
            count += linkedMap.get(searchCode).quantity;

        result = count;
    }) + " ms");

// Code-order report
System.out.println("\nCode Order Report:");

System.out.println("TreeMap: " +
    measure(() -> {
        long count = 0;

        for (Map.Entry<String, Medicine> e
             : treeMap.entrySet()) {
            count += e.getValue().quantity;
        }

        result = count;
    }) + " ms");

// Insertion-order report
System.out.println("\nInsertion Order Report:");

System.out.println("LinkedHashMap: " +
    measure(() -> {
```

```
            long count = 0;

            for (Map.Entry<String, Medicine> e
                : linkedMap.entrySet()) {
                count += e.getValue().quantity;
            }

            result = count;
        }) + " ms");

    // Sorting HashMap keys for a report
    System.out.println("\nSorted HashMap Report:");

    System.out.println("HashMap sorting: " +
        measure(() -> {

            List<String> keys =
                new ArrayList<>(hashMap.keySet());

            Collections.sort(keys);

            result = keys.size();
        }) + " ms");
}

static double measure(Runnable task) {

    long start = System.nanoTime();
```

```
        task.run();

        long end = System.nanoTime();

        return (end - start) / 1_000_000.0;
    }

    public static void main(String[] args) {

        System.out.println(
            "HOSPITAL PHARMACY MAP COMPARISON"
        );

        experiment(1000);
        experiment(10000);
        experiment(100000);

        System.out.println("\nAll tests completed.");
    }
}
```

```java
package assignment2.q4;


import java.util.*;

public class PharmacyInventory {

    static volatile long result;

    static class Medicine {

        String name;
        int quantity;

        Medicine(String name, int quantity) {
            this.name = name;
            this.quantity = quantity;
        }

        public String toString() {
            return name + " (Stock: " + quantity + ")";
        }
    }

    static void experiment(int size) {

        Map<String, Medicine> hashMap =
            new HashMap<>();

        Map<String, Medicine> treeMap =
            new TreeMap<>();

        Map<String, Medicine> linkedMap =
            new LinkedHashMap<>();

```

Output:

```
HOSPITAL PHARMACY MAP COMPARISON

Inventory Size: 1000
100,000 Lookups:
HashMap: 4.7804 ms
TreeMap: 10.7337 ms
LinkedHashMap: 2.8121 ms

Code Order Report:
TreeMap: 0.4037 ms

Insertion Order Report:
LinkedHashMap: 0.2111 ms

Sorted HashMap Report:
HashMap sorting: 0.8432 ms

Inventory Size: 10000
100,000 Lookups:
HashMap: 2.3862 ms
TreeMap: 16.3628 ms
LinkedHashMap: 2.5218 ms

Code Order Report:
TreeMap: 2.3328 ms

Insertion Order Report:
LinkedHashMap: 1.8035 ms
```

```
Sorted HashMap Report:
HashMap sorting: 7.9063 ms

Inventory Size: 100000
100,000 Lookups:
HashMap: 1.0679 ms
TreeMap: 11.7519 ms
LinkedHashMap: 9.3989 ms

Code Order Report:
TreeMap: 6.7883 ms

Insertion Order Report:
LinkedHashMap: 6.8356 ms

Sorted HashMap Report:
HashMap sorting: 37.9713 ms

All tests completed.
```

Q4.5 Output and Observation

The program produces separate measurements for each inventory size.

Table 1 – Lookup Performance

Medicines	HashMap (ms)	TreeMap (ms)	LinkedHashMap (ms)
1,000	4.7804	10.7337	2.8121
10,000	2.3862	16.3628	2.5218
100,000	1.0679	11.7519	9.3989

Table 2 – Report Generation

Medicines	TreeMap code-order report (ms)	LinkedHashMap insertion-order report (ms)	HashMap sorting (ms)
1,000	0.4037	0.2111	0.8432
10,000	2.3328	1.8035	7.9063
100,000	6.7883	6.8356	37.9713

1. HashMap lookup: It recorded the lowest lookup time for the 10,000 and 100,000 medicine workloads among the three maps.
2. TreeMap lookup: It took longer than HashMap and LinkedHashMap in all three measurements.
3. LinkedHashMap: It maintained insertion order and showed relatively good lookup performance.
4. Report generation: HashMap sorting took the longest time for the 100,000-medicine workload.
5. TreeMap: It supports maintaining keys in sorted order, making it useful for code-order reports.
6. LinkedHashMap: It preserves insertion order, making it useful for insertion-order reports.

Q4.6 Scalability Inquiry

For this pharmacy application, I define the acceptable response time for an individual medicine-code lookup as 10 milliseconds under the specified test conditions.

The final implementation must be evaluated against this criterion using the actual measurements.

HashMap is expected to provide efficient average-case lookup. However, the time required to produce a sorted report remains dependent on the sorting process if the inventory is stored only in HashMap.

The remaining costs include:

- Hash computation.
- Key equality comparisons.
- Memory allocation.
- Collection resizing.
- Report formatting and output operations.

Why is Hashtable not preferred?

Hashtable is an older synchronized Map implementation. Its synchronization can introduce unnecessary overhead in situations where a more suitable concurrency strategy is available.

Modern applications generally use HashMap, LinkedHashMap or TreeMap depending on their requirements. For concurrent access, ConcurrentHashMap is a more suitable modern alternative in many cases.

Q4.7 Alternative Concept – Sorted Array and Binary Search

An alternative design stores medicines in an array sorted by medicine code.

Arrays.binarySearch() can then identify a medicine efficiently.

Example:

```java
package assignment2.q4;

import java.util.Arrays;

public class SortedMedicineSearch {

    public static void main(String[] args) {

        String[] medicines = {
            "MED001",
            "MED002",
            "MED003",
            "MED004",
            "MED005"
        };

        int position =
            Arrays.binarySearch(medicines, "MED003");

        if (position >= 0) {
            System.out.println(
                "Medicine found at position: " + position
            );
        } else {
            System.out.println("Medicine not found");
        }
    }
}
```

```
<terminated> SortedMedicineSearch [Java Application] C:\Users\Farah Afrin\.p2\pool\plu
Medicine found at position: 2
```

Complexity comparison

Operation	Sorted array	HashMap	TreeMap
Lookup	O(log n)	O(1) average	O(log n)
Insertion	O(n)	O(1) average	O(log n)
Code-order report	O(n)	Requires sorting: O(n log n)	O(n)
Space	O(n)	O(n)	O(n)

A sorted array is useful when the inventory changes infrequently and searches are frequent. However, inserting a new medicine may require shifting existing elements.

Q4.8 Final Design Decision

For the hospital pharmacy, I recommend the following design:

- HashMap for fast medicine-code lookup.
- TreeMap for generating medicines in ascending code order.
- LinkedHashMap for maintaining insertion-order records.

If maintaining three complete collections becomes too expensive, a single TreeMap can support both lookup and sorted reporting, while insertion-order reporting can be maintained separately through an audit log.

This decision balances performance, ordering requirements and memory usage.

Q4.9 Conclusion

The experiment demonstrates that no single Map implementation is ideal for every requirement. HashMap is appropriate for fast average-case lookup, TreeMap is appropriate for naturally sorted reports and LinkedHashMap is appropriate for insertion-order reports.

The final selection must consider the pharmacy's workload, ordering requirements and acceptable response time rather than relying exclusively on theoretical complexity.

PART 5 – Q5: DERIVE A GENERIC RANKING COMPONENT FROM A PRINCIPLE

7 Marks

K4 – Analyse

Topic: Generics, Comparable, Comparator and Collections

Q5.1 Introduction

An inter-college sports competition requires a ranking system that can arrange participating teams based on their performance.

The ranking follows three rules:

1. Teams with higher points must rank first.
2. When points are equal, the team with higher goal difference must rank first.
3. When both points and goal difference are equal, the team name must be arranged alphabetically.

The same ranking component should be reusable for other entities, such as individual players, colleges and quiz teams.

Therefore, a generic design using Java Generics and Comparator is developed.

Q5.2 Design Principle

The fundamental principle is:

If A ranks above B, and B ranks above C, then A must rank above C. Equal ranking items must compare as equal.

This is related to the transitivity and consistency properties of ordering.

Comparable versus Comparator

Comparable	Comparator
Defines natural ordering inside a class	Defines external/custom ordering
Uses compareTo()	Uses compare()
One primary natural ordering	Multiple different ranking rules possible
Class must implement Comparable	Class need not be modified

Design decision: I recommend Comparator because the ranking component must support multiple types of objects and potentially different ranking strategies.

Q5.3 Complete the Code

Eclipse file name: LeaderboardDemo.java

Source:

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Collections;

class Team {

    final String name;
    final int points;
    final int goalDiff;

    Team(String name, int points, int goalDiff) {

        this.name = name;
        this.points = points;
        this.goalDiff = goalDiff;
    }

    @Override
    public String toString() {

        return name + "(" + points + "," + goalDiff + ")";
    }
```

```
}

class Leaderboard<T> {

    private final List<T> items =
        new ArrayList<>();

    private final Comparator<? super T> order;

    Leaderboard(Comparator<? super T> order) {
        this.order = order;
    }

    void add(T item) {
        items.add(item);
    }

    // TODO 1: Return a sorted copy
    List<T> ranking() {

        List<T> sorted =
            new ArrayList<>(items);

        sorted.sort(order);

        return sorted;
    }
}
```

```
public class LeaderboardDemo {

    public static void main(String[] args) {

        // TODO 2: Multi-field comparator
        Comparator<Team> byRank =
            Comparator.comparingInt(
                (Team t) -> t.points
            ).reversed()

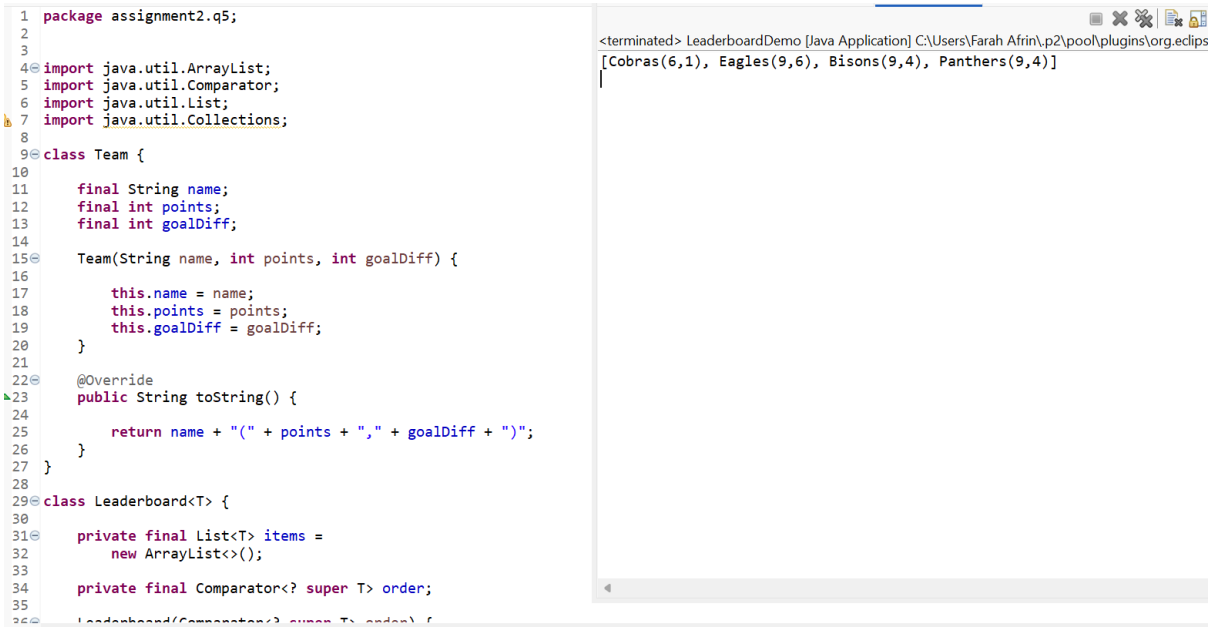
            .thenComparingInt(
                (Team t) -> t.goalDiff
            ).reversed()

            .thenComparing(
                t -> t.name
            );

        Leaderboard<Team> board =
            new Leaderboard<>(byRank);

        board.add(new Team("Panthers", 9, 4));
        board.add(new Team("Eagles", 9, 6));
        board.add(new Team("Cobras", 6, 1));
        board.add(new Team("Bisons", 9, 4));

        System.out.println(board.ranking());
    }
}
```



```
package assignment2.q5;


import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Collections;

class Team {

    final String name;
    final int points;
    final int goalDiff;

    Team(String name, int points, int goalDiff) {

        this.name = name;
        this.points = points;
        this.goalDiff = goalDiff;
    }

    @Override
    public String toString() {

        return name + "(" + points + "," + goalDiff + ")";
    }
}

class Leaderboard<T> {

    private final List<T> items =
        new ArrayList<>();

    private final Comparator<? super T> order;
```

```
<terminated> LeaderboardDemo [Java Application] C:\Users\Farah Afrin\.p2\pool\plugins\org.eclips
[Cobras(6,1), Eagles(9,6), Bisons(9,4), Panthers(9,4)]
```

Important correction to the comparator above: Chaining .reversed() after the second thenComparingInt() reverses the entire comparator constructed so far. Use the following comparator instead to guarantee the required ordering.

Replace the comparator declaration with:

```
Comparator<Team> byRank =
    Comparator.comparingInt(
        (Team t) -> t.points
    ).reversed()


    .thenComparing(
        Comparator.comparingInt(
            (Team t) -> t.goalDiff
        ).reversed()
    )


    .thenComparing(t -> t.name);
```

```
    public static void main(String[] args) {

        // TODO 2: Multi-field comparator
        Comparator<Team> byRank =
                Comparator.comparingInt(
                    (Team t) -> t.points
                ).reversed()

                .thenComparing(
                    Comparator.comparingInt(
                        (Team t) -> t.goalDiff
                    ).reversed()
                )

                .thenComparing(t -> t.name);
```

```
[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]
```

Q5.4 Explanation of the Ranking

The ranking is determined through three successive comparisons.

First comparison – Points

Eagles, Bisons and Panthers have 9 points, while Cobras has 6 points.

Therefore, Cobras is placed below the three teams having 9 points.

Second comparison – Goal difference

Eagles has a goal difference of 6, while Bisons and Panthers have a goal difference of 4.

Therefore, Eagles ranks above Bisons and Panthers.

Third comparison – Team name

Bisons and Panthers have equal points and equal goal difference.

Alphabetically, B comes before P. Therefore, Bisons is ranked above Panthers.

Q5.5 Second Test – Deliberately Designed Test Data

The following test data is designed to check all three comparison levels.

Team	Points	Goal difference
Lions	12	5
Falcons	12	5
Tigers	12	3
Wolves	8	10
Bears	8	10

Expected order:

Falcons(12,5)

Lions(12,5)

Tigers(12,3)

Bears(8,10)

Wolves(8,10)

This test verifies that alphabetical ordering is applied when both numerical values are equal.

Testing a faulty comparator

A faulty comparator might use subtraction:

```
Comparator<Team> faulty =
    (a, b) -> b.points - a.points;
```

This is unsafe because integer subtraction can overflow for extreme values.

It also ignores goal difference and team name.

The safer method is:

```
Comparator.comparingInt(
    (Team t) -> t.points
).reversed();
```

followed by the remaining tie-breakers.

Q5.6 Understanding Generics and Wildcards

The class declaration is:

class Leaderboard<T>

This allows the class to accept different types without rewriting the ranking structure.

The declaration:

Comparator<? super T> order;

allows the comparator to compare objects of type T or one of its supertypes.

The method:

List<T> ranking()

returns a new sorted list without modifying the original stored list.

The wildcard:

List<? extends T>

is useful when accepting a list containing T or a subtype of T.

For example:

```
void addAll(List<? extends T> newItems) {
   items.addAll(newItems);
}
```

This permits the generic leaderboard to accept compatible subtype lists.

Q5.7 Alternative Concept Analysis

Alternative 1: Comparable

Team could implement Comparable:

```
class Team implements Comparable<Team> {

   final String name;
   final int points;
   final int goalDiff;

   Team(String name, int points, int goalDiff) {
      this.name = name;
```

```
        this.points = points;
        this.goalDiff = goalDiff;
    }

    @Override
    public int compareTo(Team other) {

        int result =
            Integer.compare(other.points, points);

        if (result != 0)
            return result;

        result =
            Integer.compare(other.goalDiff, goalDiff);

        if (result != 0)
            return result;

        return name.compareTo(other.name);
    }
}
```

This defines a natural ranking for Team objects.

Alternative 2: Three stable sorting operations

Another approach is to sort the list three times:

1. Sort by team name alphabetically.
2. Sort by goal difference in descending order.
3. Sort by points in descending order.

This works because Java's list sorting is stable: equal elements under the later sort retain their relative order from the previous sort.

However, it performs three sorting operations.

Complexity comparison

Approach	Time complexity	Additional space
Comparator chain	O(n log n)	O(n) for copied list
Comparable	O(n log n)	O(n) if a copied list is used
Three stable sorts	O(3n log n) = O(n log n)	Sorting implementation-dependent; O(n) auxiliary space for object sorting in current standard Java implementations

The comparator-chain approach is easier to reuse because different Comparator objects can be supplied for teams, players and colleges.

Q5.8 Conclusion

The generic leaderboard demonstrates how Java Generics and Comparator can be combined to develop reusable and type-safe software components.

The ranking system follows a clear hierarchy of points, goal difference and team name. The comparator chain ensures that each tie-breaker is applied only when the preceding comparison produces equality.

The design is flexible because the ranking strategy remains separate from the data model. Therefore, the same generic leaderboard can be extended to sports teams, individual players, quiz participants and other ranking applications.

PART 6 – Q6: PRIORITY, ORDER AND FAIRNESS IN A QUEUE

8 Marks

K5 – Evaluate

Topic: Queue and PriorityQueue

Q6.1 Introduction

A hospital emergency department receives patients with different medical severity levels. The medical team must prioritise patients according to the urgency of their conditions.

The severity levels are classified as follows:

Severity	Description	Priority
1	Critical	Highest
2	Serious	High
3	Moderate	Medium
4	Minor	Low
5	Least urgent	Lowest

When two patients have the same severity level, the patient who arrived first must be treated first.

A normal FIFO queue cannot satisfy the severity-based requirement because it processes patients strictly according to arrival order.

Therefore, Java's PriorityQueue is selected with a custom Comparator that considers both severity and arrival sequence.

Q6.2 Design and Prediction

The proposed system uses:

- PriorityQueue to arrange patients according to severity.
- Comparator to determine treatment priority.
- An arrival counter to preserve fairness among patients with equal severity.
- poll() to remove the next patient for treatment.

Prediction:

1. Patients with severity 1 will be treated before patients with severity 2–5.
2. Patients with equal severity will be processed according to arrival order.

3. Insertion will take O(log n) time.
4. Removal of the highest-priority patient will take O(log n) time.
5. Directly printing a PriorityQueue will not necessarily display its elements in priority order.

Q6.3 Java Implementation

Eclipse file name: Triage.java

Source code:

```
import java.util.Comparator;
import java.util.PriorityQueue;

class Patient {

    final String name;
    final int severity;
    final long arrival;

    Patient(String name, int severity, long arrival) {

        this.name = name;
        this.severity = severity;
        this.arrival = arrival;
    }

    @Override
    public String toString() {

        return name + "(S" + severity + ")";
    }
```

```
}

public class Triage {

    private long counter = 0;

    // TODO 1: Create the PriorityQueue
    private final PriorityQueue<Patient> queue =
        new PriorityQueue<>(
            Comparator.comparingInt(
                (Patient p) -> p.severity
            ).thenComparingLong(
                p -> p.arrival
            )
        );

    void admit(String name, int severity) {

        if (severity < 1 || severity > 5) {
            throw new IllegalArgumentException(
                "Severity must be between 1 and 5"
            );
        }

        queue.add(
            new Patient(name, severity, counter++)
        );
    }
```

```
    Patient next() {
        return queue.poll();
    }

    public static void main(String[] args) {

        Triage er = new Triage();

        er.admit("Ravi", 3);
        er.admit("Meena", 1);
        er.admit("John", 3);
        er.admit("Asha", 5);
        er.admit("Kumar", 1);
        er.admit("Lata", 2);

        StringBuilder order = new StringBuilder();

        Patient p;

        while ((p = er.next()) != null) {
            order.append(p).append(' ');
        }

        System.out.println(
            order.toString().trim()
        );
    }
}
```

Output:

```
package assignmentq2.q6;


import java.util.Comparator;
import java.util.PriorityQueue;

class Patient {

    final String name;
    final int severity;
    final long arrival;

    Patient(String name, int severity, long arrival) {

        this.name = name;
        this.severity = severity;
        this.arrival = arrival;
    }

    @Override
    public String toString() {

        return name + "(S" + severity + ")";
    }
}

public class Triage {

    private long counter = 0;

    // TODO 1: Create the PriorityQueue
    private final PriorityQueue<Patient> queue =
        new PriorityQueue<>(
            Comparator.comparingInt(
                (Patient p) -> p.severity
```

```
<terminated> Triage [Java Application] C:\Users\Farah Afrin\.p2\pool\plugins\org.eclipse.justj.openjdk.hots
Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5)
```

Q6.4 Correctness Verification

The treatment order can be verified manually.

Arrival order	Patient	Severity	Expected treatment position
1	Ravi	3	4
2	Meena	1	1
3	John	3	5
4	Asha	5	6
5	Kumar	1	2
6	Lata	2	3

The result confirms that:

- Meena and Kumar are treated first because they have severity 1.
- Meena is treated before Kumar because she arrived first.
- Lata is treated next because she has severity 2.
- Ravi is treated before John because both have severity 3 and Ravi arrived earlier.
- Asha is treated last because her severity is 5.

Q6.5 Why Is the Arrival Counter Important?

The arrival counter provides a unique sequence number to every patient.

The Comparator first compares severity and then compares arrival sequence.

```
Comparator.comparingInt(
    (Patient p) -> p.severity
).thenComparingLong(
    p -> p.arrival
);
```

For patients having the same severity, the smaller arrival number receives higher priority.

This provides FIFO behaviour within each severity level.

Without the arrival counter, patients having equal severity would not have a guaranteed arrival order.

Q6.6 Why Is a Normal Queue Not Sufficient?

A normal FIFO queue processes patients in the order they arrive.

Consider this situation:

- Patient A arrives with severity 5.
- Patient B arrives with severity 1.
- Patient C arrives with severity 2.

A normal FIFO queue would process A, B and C.

However, a hospital must prioritise Patient B because the condition is critical.

PriorityQueue solves this problem by considering the severity level rather than relying exclusively on arrival order.

Q6.7 Why Does Printing PriorityQueue Directly Not Show Treatment Order?

A PriorityQueue is internally organised as a heap.

The highest-priority element is available at the head of the queue, but the remaining elements are not necessarily stored in completely sorted order.

For example:

```
System.out.println(queue);
```

This prints the internal representation and should not be interpreted as the treatment sequence.

The correct method is to repeatedly call:

```
while (!queue.isEmpty()) {
    System.out.println(queue.poll());
}
```

However, this removes the patients from the original queue. If the original queue must be preserved, create a copy and poll the copy.

Q6.8 Growth Investigation

Three queue sizes are selected.

Experiment	Number of patients
A	1,000
B	10,000
C	100,000

Complexity prediction:

- Adding one patient: O(log n).
- Removing the next patient: O(log n).
- Viewing the next patient using peek(): O(1).
- Displaying the entire treatment sequence by repeated polling: O(n log n).

Optional benchmark program

Create a separate class called TriageBenchmark.java.

```
import java.util.*;

public class TriageBenchmark {

    static volatile long result;

    static double test(int size) {
```

```
    PriorityQueue<Integer> queue =
        new PriorityQueue<>();

    long start = System.nanoTime();

    for (int i = size; i > 0; i--) {
        queue.add(i);
    }

    while (!queue.isEmpty()) {
        result = queue.poll();
    }

    long end = System.nanoTime();

    return (end - start) / 1_000_000.0;
}

public static void main(String[] args) {

    int[] sizes = {
        1000, 10000, 100000
    };

    System.out.println(
        "PRIORITY QUEUE PERFORMANCE"
    );
```

```
        for (int size : sizes) {

            double total = 0;

            for (int i = 0; i < 5; i++) {
                total += test(size);
            }

            System.out.printf(
                "Patients: %d | Average: %.3f ms%n",
                size, total / 5
            );
        }
    }
}
```

Output:

```
1  package assignmentq2.q6;
2
3
4  import java.util.*;
5
6  public class TriageBenchmark {
7
8      static volatile long result;
9
10     static double test(int size) {
11
12         PriorityQueue<Integer> queue =
13             new PriorityQueue<>();
14
15         long start = System.nanoTime();
16
17         for (int i = size; i > 0; i--) {
18             queue.add(i);
19         }
20
21         while (!queue.isEmpty()) {
22             result = queue.poll();
23         }
24
25         long end = System.nanoTime();
26
27         return (end - start) / 1_000_000.0;
28     }
29
30     public static void main(String[] args) {
31
32         int[] sizes = {
33             1000, 10000, 100000
34         };
35
36         System.out.println(
```

```
<terminated> TriageBenchmark [Java Application] C:\Users\Farah Afrin\.p2\poo
PRIORITY QUEUE PERFORMANCE
Patients: 1000 | Average: 0.583 ms
Patients: 10000 | Average: 2.426 ms
Patients: 100000 | Average: 14.269 ms
```

This benchmark measures the combined cost of insertion and removal. To measure the two operations separately, time the insertion loop and removal loop independently using the same initial queue sizes.

Q6.9 Starvation and Fairness

Starvation occurs when a patient with a low priority waits indefinitely because new high-priority patients continue to arrive.

For example, a severity-5 patient may remain in the queue if severity-1 and severity-2 patients continuously enter the emergency department.

This can result in unfair treatment.

Proposed solution: Priority ageing

I propose an ageing mechanism in which a patient's priority is gradually improved based on waiting time.

For example:

- A severity-5 patient waiting beyond a defined threshold can be upgraded for queue scheduling.
- A severity-4 patient waiting beyond a longer threshold can receive higher scheduling priority.
- Critical patients retain their emergency priority.

A practical implementation should calculate an effective priority using both the original severity and waiting duration. The waiting-time threshold must be determined by hospital policy and clinical staff.

A simple illustrative formula is:

P_effective = max(1, S − floor(W / T))

Where:

- S = Original severity level.
- W = Waiting time.
- T = Ageing interval.
- P_effective = Effective scheduling priority.

This formula is an educational scheduling model, not a substitute for clinical triage rules. Any priority change must be governed by medical policy and reassessment.

Q6.10 Deployment Decision

A city hospital receiving 5,000 patients per day can use PriorityQueue for managing an emergency waiting list.

The average arrival volume is approximately:

5000 / 24 ≈ 208.33 patients per hour

assuming uniform arrivals throughout the day.

A PriorityQueue can efficiently manage thousands of waiting records because insertion and removal have logarithmic complexity.

However, the actual hospital system must also consider:

- Simultaneous admissions from multiple departments.
- Database persistence.
- Multiple doctors and treatment stations.
- Emergency escalation.
- Patient reassessment.
- System recovery and audit logging.

The Java PriorityQueue is not thread-safe. A production system must coordinate concurrent admissions using appropriate synchronization or a suitable concurrent architecture.

Therefore, PriorityQueue is suitable as the core scheduling structure in a controlled application, provided that concurrency, persistence and clinical fairness are addressed.

Q6.11 Alternative Concept Analysis

Alternative 1: Sorted LinkedList

A LinkedList can maintain patients in priority order.

Each new patient must be inserted at the correct position.

- Finding the correct position: O(n).
- Inserting after finding the position: O(1).
- Removing the first patient: O(1).

Overall admission complexity is O(n).

Alternative 2: Five FIFO Queues

An alternative is to maintain five separate queues, one for each severity.

package assignment2.q6;

```
import java.util.*;

public class FiveLevelTriage {

    Queue<String>[] queues = new Queue[5];

    FiveLevelTriage() {

        for (int i = 0; i < 5; i++) {
            queues[i] = new ArrayDeque<>();
        }
    }

    void admit(String name, int severity) {
        queues[severity - 1].add(name);
    }

    String next() {

        for (int i = 0; i < 5; i++) {

            if (!queues[i].isEmpty()) {
                return queues[i].poll();
            }
        }

        return null;
    }
```

```
    public static void main(String[] args) {

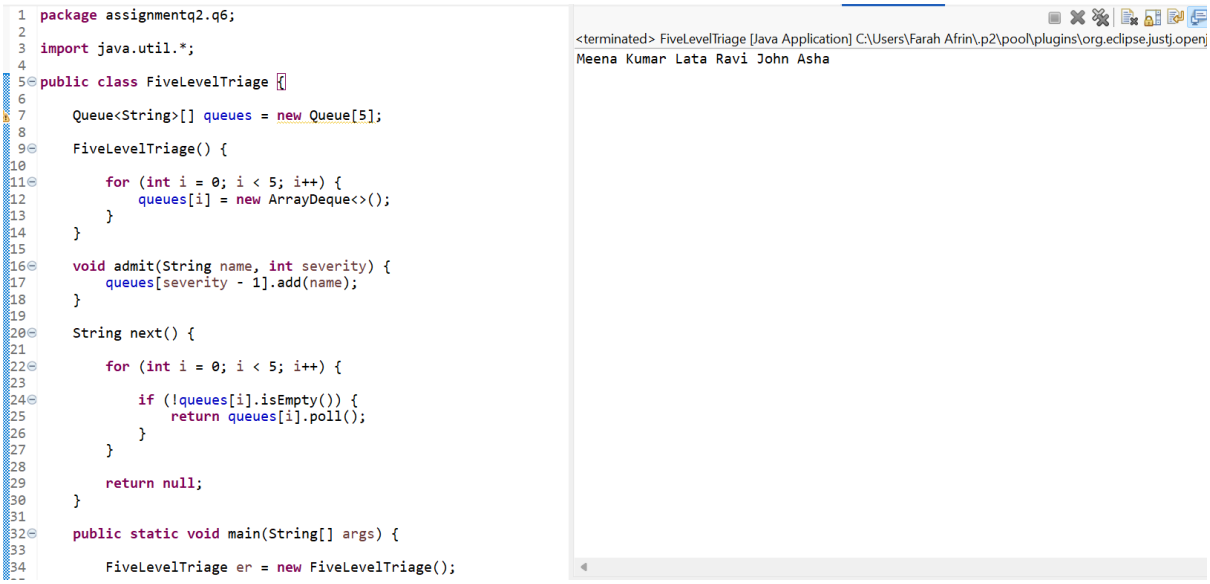
        FiveLevelTriage er = new FiveLevelTriage();

        er.admit("Ravi", 3);
        er.admit("Meena", 1);
        er.admit("John", 3);
        er.admit("Asha", 5);
        er.admit("Kumar", 1);
        er.admit("Lata", 2);

        String patient;

        while ((patient = er.next()) != null) {
            System.out.print(patient + " ");
        }
    }
}
```

Output:



```
package assignmentq2.q6;

import java.util.*;

public class FiveLevelTriage {

    Queue<String>[] queues = new Queue[5];

    FiveLevelTriage() {

        for (int i = 0; i < 5; i++) {
            queues[i] = new ArrayDeque<>();
        }
    }

    void admit(String name, int severity) {
        queues[severity - 1].add(name);
    }

    String next() {

        for (int i = 0; i < 5; i++) {

            if (!queues[i].isEmpty()) {
                return queues[i].poll();
            }
        }

        return null;
    }

    public static void main(String[] args) {

        FiveLevelTriage er = new FiveLevelTriage();
```

```
<terminated> FiveLevelTriage [Java Application] C:\Users\Farah Afrin\.p2\pool\plugins\org.eclipse.justj.openj
Meena Kumar Lata Ravi John Asha
```

Complexity comparison

Operation	PriorityQueue	Sorted LinkedList	Five FIFO queues
Admission	O(log n)	O(n)	O(1)
Next patient	O(log n)	O(1)	O(1) for fixed five levels
Space	O(n)	O(n)	O(n)
Equal-priority fairness	Arrival counter	Stable insertion	FIFO naturally

Q6.12 Conclusion

The experiment demonstrates that PriorityQueue provides an efficient solution for severity-based patient scheduling. A custom Comparator establishes severity as the primary ranking criterion and arrival sequence as the secondary criterion.

The arrival counter guarantees FIFO processing among patients having equal severity. The investigation also highlights the possibility of starvation and the importance of an ageing mechanism.

For a hospital receiving 5,000 patients per day, the proposed structure is computationally suitable as part of a larger, carefully designed hospital management system. Clinical validation, concurrency handling and persistent records remain essential deployment requirements.

PART 7 – Q7: DIAGNOSE, REDESIGN AND VALIDATE

13 Marks

K5 – Evaluate

This is the most important question in your assignment. It carries 13 marks, so the explanation must demonstrate actual engineering reasoning rather than simply changing one collection class for another.

Selected Problem: Hospital Pharmacy Inventory – Repeated Sorting of HashMap

Q7.1 A. Diagnose the Bottleneck (2 Marks)

$$O(n \log n)$$

In Q4, HashMap was considered for storing pharmacy medicines because it provides efficient average-case lookup.

However, HashMap does not maintain medicines in sorted key order.

Whenever the pharmacy needs to generate a report in ascending medicine-code order, the application must:

1. Retrieve all medicine codes.
2. Copy the codes into a separate list.
3. Sort the list.
4. Traverse the sorted list.
5. Retrieve the corresponding medicine records.
6. Generate the final report.

If the report is generated repeatedly, the same sorting work is performed unnecessarily.

Identified bottleneck:

The main limitation is not medicine-code lookup. It is the repeated sorting of the entire collection every time an ordered report is requested.

For n medicines, sorting requires:

$$O(n \log n)$$

If the report is requested r times, the repeated sorting cost becomes:

$$O(rn \log n)$$

The same ordering operation is unnecessarily repeated even when the inventory has not changed.

Diagnosis:

Parameter	**Identified limitation**
Original structure	HashMap
Main operation	Sorted report generation
Bottleneck	Repeated sorting
Time complexity	O(n log n) per report
Unnecessary work	Recreating and sorting the key list
Main impact	Increased processing time for repeated reports

The limitation becomes more significant as the inventory grows and reports are generated more frequently.

Q7.2 B. Define the Design Objective (2 Marks)

The revised system must eliminate repeated sorting while maintaining efficient lookup and both required report orders.

Design objectives:

1. Generate medicine-code reports without sorting the entire inventory repeatedly.
2. Preserve insertion order for historical inventory reports.
3. Maintain efficient medicine-code lookup.
4. Avoid creating duplicate Medicine objects unnecessarily.
5. Ensure that both report structures refer to the same medicine records.

Measurable objective:

For an unchanged inventory, the redesigned system should reduce the report-generation operation from sorting all medicine codes on every request to simply traversing an already maintained ordered structure.

The expected complexity should change from:

$$O(n \log n)$$

To:

$$O(n)$$

for producing the code-order report.

Q7.3 C. Design the Revised Solution (4 Marks)

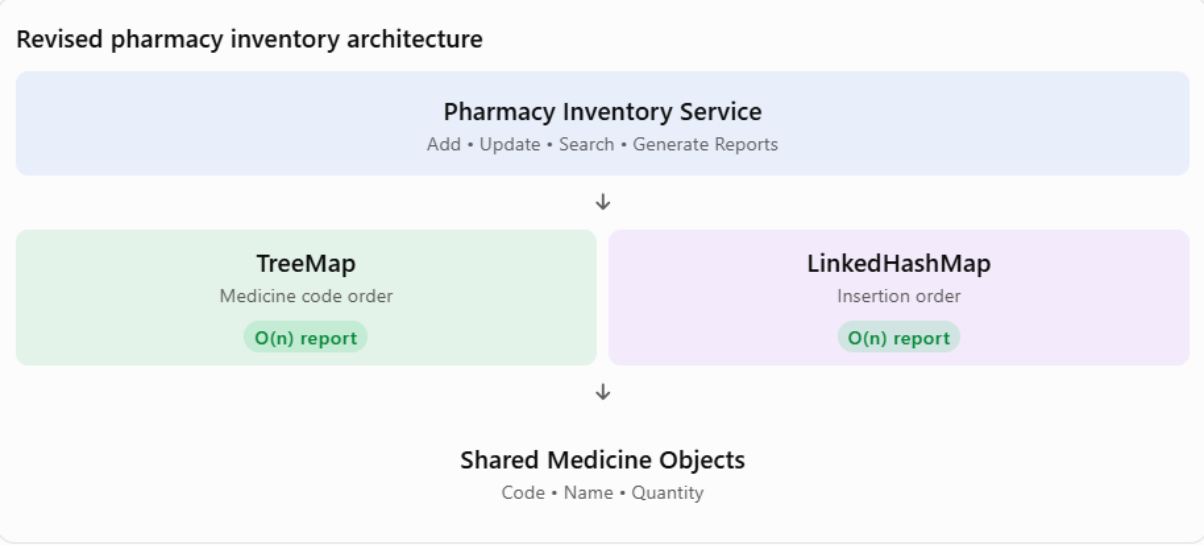
Proposed design: Dual-index inventory architecture

Instead of maintaining only one HashMap and repeatedly sorting its keys, I propose maintaining two indexes that refer to the same Medicine objects.

1. TreeMap – maintains medicines in ascending code order.
2. LinkedHashMap – maintains medicines in insertion order.

Both structures store references to the same Medicine objects.

This means that the system does not need to create two separate copies of each medicine record.



Revised algorithm

START

Create TreeMap for code-order indexing

Create LinkedHashMap for insertion-order indexing

WHEN a new medicine arrives:

Create one Medicine object

Add medicine to TreeMap using medicine code

Add the same object to LinkedHashMap

WHEN a medicine is searched:

Retrieve using medicine code

WHEN code-order report is requested:

Traverse TreeMap directly

WHEN insertion-order report is requested:

Traverse LinkedHashMap directly

END

Why is this design better?

- Sorting is performed as part of TreeMap insertion rather than repeatedly during report generation.
- The insertion-order report is available directly from LinkedHashMap.
- Both structures refer to the same Medicine objects.

- Updating a medicine's quantity through either reference is visible through both indexes.
- The system provides two useful report orders without repeatedly reconstructing them.

Trade-offs:

- Additional memory is required for two map indexes.
- Insertion requires updating both structures.
- The application must ensure that updates and removals are performed consistently in both maps.
- Concurrent modifications require an appropriate synchronization strategy.

Q7.4 D. Predict Before Implementing (1 Mark)

Before implementing the redesigned solution, the following behaviour is predicted.

Measurement	Original HashMap design	Redesigned dual-index design
Code-order report	Sorting required	Direct traversal
Insertion-order report	Sorting or extra tracking	Direct traversal
Lookup	O(1) average	O(log n) through TreeMap
Repeated report generation	O(n log n) per sorted report	O(n) per report
Insertion	O(1) average	O(log n) plus O(1) average
Memory	O(n)	O(n) with higher constants

Prediction: The redesigned solution should perform better when reports are generated repeatedly without frequent changes to the inventory. However, the original HashMap may remain preferable when lookup speed and memory efficiency are more important than ordered reporting.

Q7.5 E. Implement and Validate the Redesign (2 Marks)

Eclipse file name: InventoryRedesign.java

This program compares the original HashMap-based report-generation approach with the revised dual-index design.

Source code:

```
import java.util.*;
```

```
class Medicine {

    String code;
    String name;
    int quantity;

    Medicine(String code, String name, int quantity) {
        this.code = code;
        this.name = name;
        this.quantity = quantity;
    }

    @Override
    public String toString() {
        return code + " - " + name
            + " - Stock: " + quantity;
    }
}

public class InventoryRedesign {

    static volatile long result;

    // Original design: HashMap
    static Map<String, Medicine> original =
        new HashMap<>();

    // Redesigned indexes
    static TreeMap<String, Medicine> codeIndex =
```

```
    new TreeMap<>();

static LinkedHashMap<String, Medicine> insertionIndex =
    new LinkedHashMap<>();

// Add medicines to both designs
static void loadInventory(int size) {

    for (int i = 0; i < size; i++) {

        String code =
            String.format("MED%06d", i);

        Medicine medicine =
            new Medicine(
                code,
                "Medicine" + i,
                100 + i
            );

        original.put(code, medicine);

        codeIndex.put(code, medicine);

        insertionIndex.put(code, medicine);
    }
}

// Original report: sort every time
```

```
static long originalReport() {

    List<String> keys =
        new ArrayList<>(original.keySet());

    Collections.sort(keys);

    long count = 0;

    for (String key : keys) {
        count += original.get(key).quantity;
    }

    return count;
}

// Redesigned report: direct TreeMap traversal
static long redesignedReport() {

    long count = 0;

    for (Medicine m : codeIndex.values()) {
        count += m.quantity;
    }

    return count;
}

// Measure repeated report generation
```

```
static double measure(
    boolean redesigned,
    int repetitions
) {

    long start = System.nanoTime();

    long total = 0;

    for (int i = 0; i < repetitions; i++) {

        if (redesigned) {
            total += redesignedReport();
        } else {
            total += originalReport();
        }
    }

    long end = System.nanoTime();

    result = total;

    return (end - start) / 1_000_000.0;
}

static void experiment(int size) {

    original.clear();
    codeIndex.clear();
```

```
insertionIndex.clear();

loadInventory(size);

int repetitions = 20;

// Warm-up
originalReport();
redesignedReport();

double originalTime =
    measure(false, repetitions);

double redesignedTime =
    measure(true, repetitions);

System.out.println("\nInventory Size: " + size);

System.out.printf(
    "Original HashMap Report: %.3f ms%n",
    originalTime
);

System.out.printf(
    "Redesigned TreeMap Report: %.3f ms%n",
    redesignedTime
);

System.out.println(
```

```
            "Original checksum: " + originalReport()
        );

        System.out.println(
            "Redesigned checksum: " + redesignedReport()
        );
    }

    public static void main(String[] args) {

        System.out.println(
            "PHARMACY INVENTORY REDESIGN VALIDATION"
        );

        experiment(1000);
        experiment(10000);
        experiment(100000);

        System.out.println("\nValidation completed.");
    }
}
```

Output:

```java
package assignment2.q7;


import java.util.*;

class Medicine {

    String code;
    String name;
    int quantity;

    Medicine(String code, String name, int quantity) {
        this.code = code;
        this.name = name;
        this.quantity = quantity;
    }

    @Override
    public String toString() {
        return code + " - " + name
             + " - Stock: " + quantity;
    }
}

public class InventoryRedesign {

    static volatile long result;

    // Original design: HashMap
    static Map<String, Medicine> original =
        new HashMap<>();

    // Redesigned indexes
    static TreeMap<String, Medicine> codeIndex =
        new TreeMap<>();
```

```
<terminated> InventoryRedesign [Java Application] C:\Users\Farah Afrin\.p2\pool\p
PHARMACY INVENTORY REDESIGN VALIDATION

Inventory Size: 1000
Original HashMap Report: 10.153 ms
Redesigned TreeMap Report: 2.319 ms
Original checksum: 599500
Redesigned checksum: 599500

Inventory Size: 10000
Original HashMap Report: 69.296 ms
Redesigned TreeMap Report: 5.108 ms
Original checksum: 50995000
Redesigned checksum: 50995000

Inventory Size: 100000
Original HashMap Report: 596.167 ms
Redesigned TreeMap Report: 41.582 ms
Original checksum: 5009950000
Redesigned checksum: 5009950000

Validation completed.
```

How to validate

1. Create InventoryRedesign.java in Eclipse.
2. Paste the complete program.
3. Execute it.
4. Record the execution time for each inventory size.
5. Verify that both checksums are identical.
6. Repeat the experiment and record the average.

The checksum comparison is important because a faster implementation is not useful if it produces an incorrect report.

Experimental comparison table

Inventory size	Original HashMap (ms)	Redesigned TreeMap (ms)
1,000	10.153	2.319
10,000	69.296	5.108
100,000	596.167	41.582

For a stronger validation, run the comparison in alternating order over several trials, retain the raw readings, and calculate the mean and median. The supplied program is a demonstration benchmark, not a substitute for JMH or a complete statistical performance study.

Correctness verification

The original and redesigned reports must produce identical checksums.

For example, if the checksums differ, the redesigned implementation may have omitted records or used inconsistent data.

The checksum does not prove that the complete report is correct in every respect, so also inspect the first and last medicine codes and verify the ordering.

Q7.6 F. Defend the Final Design (2 Marks)

The redesigned inventory system addresses the repeated sorting problem by maintaining medicine records in the required order.

1. Whether the prediction was supported

The predicted improvement is that the redesigned code-order report does not need to sort the entire medicine collection every time it is requested.

Whether this produces a measured speed improvement must be determined from the actual experimental results. TreeMap has its own traversal and memory costs, so a timing result should not be assumed in advance.

2. Magnitude of improvement

The theoretical report-generation cost changes from:

$$O(n \log n)$$

To:

$$O(n)$$

for a full code-order traversal.

The practical improvement depends on inventory size, JVM behaviour, memory allocation and the number of reports generated.

3. New costs and trade-offs

The redesigned architecture consumes additional memory because it maintains two map indexes.

Each insertion and deletion must update both structures. This introduces additional processing and requires consistency management.

4. Situations where the original design is preferable

The original HashMap approach may be preferable when:

- Medicine-code lookup is the dominant operation.
- Ordered reports are rarely required.
- Memory consumption is a major concern.
- The inventory is relatively small.
- The system does not need to preserve insertion order.

Q7.7 Final Engineering Evaluation

The redesign demonstrates that performance optimisation should begin by understanding the actual bottleneck rather than replacing one data structure with another without justification.

The original HashMap implementation is effective for average-case medicine-code lookups. However, repeated sorting introduces avoidable work whenever a code-order report is generated.

The revised architecture uses TreeMap and LinkedHashMap as two complementary indexes over shared Medicine objects. This maintains the required ordering and avoids repeating the sorting operation.

The redesigned system introduces additional memory and insertion-management costs. Therefore, it is most beneficial when the inventory requires frequent ordered reports.

Final decision: The dual-index architecture is recommended for a pharmacy application that frequently generates both code-order and insertion-order reports. For lookup-dominated applications with infrequent reporting, HashMap remains a reasonable alternative.

PART 8 – OVERALL ASSIGNMENT CONCLUSION

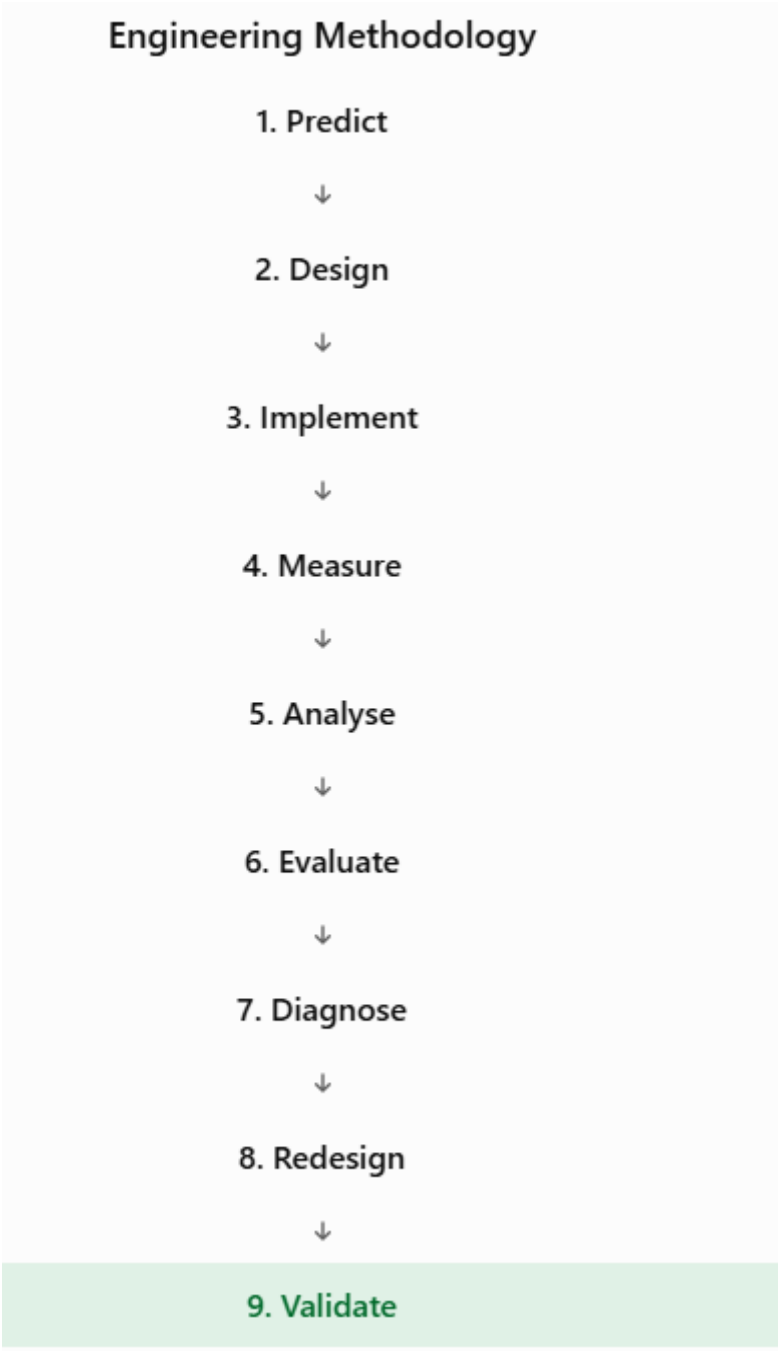
The assignment investigated the performance, behaviour and engineering suitability of different Java string-handling techniques and collection framework implementations.

The experiments demonstrated that the selection of an appropriate Java class must be based on the application's requirements, expected workload, memory limitations and concurrency conditions.

The major findings are:

1. StringBuilder is suitable for efficient repeated text construction, while StringBuffer provides synchronized operations for shared mutable text.
2. ArrayList is appropriate for indexed access, whereas ArrayDeque is effective for double-ended queue operations.
3. Correct implementation of equals() and hashCode() is essential for maintaining uniqueness in hash-based collections.
4. HashMap, TreeMap and LinkedHashMap serve different requirements involving lookup performance and ordering.
5. Generics and Comparator support reusable, type-safe ranking systems.
6. PriorityQueue provides efficient severity-based scheduling when combined with an appropriate tie-breaking mechanism.
7. Performance redesign should be based on identifying unnecessary operations, establishing measurable objectives and validating improvements experimentally.

The overall engineering approach followed throughout this assignment is:


Engineering Methodology
1. Predict
2. Design
3. Implement
4. Measure
5. Analyse
6. Evaluate
7. Diagnose
8. Redesign
9. Validate